Fundamentals of API Development

API Lifecycle: Overview & Versioning

Day 3 - Session 1

KavinSchool

Day 2 Recap

From yesterday	Feeds today
A peer-reviewed OpenAPI spec with interactive docs	Today's versioning strategy needs a stable contract to version
Contract-first vs code-first tradeoffs	Another sequencing tradeoff: design lifecycle up front, or let it emerge
springdoc-openapi and Swagger UI	Where the version number itself will show up (`info.version`, the path)

What We'll Cover

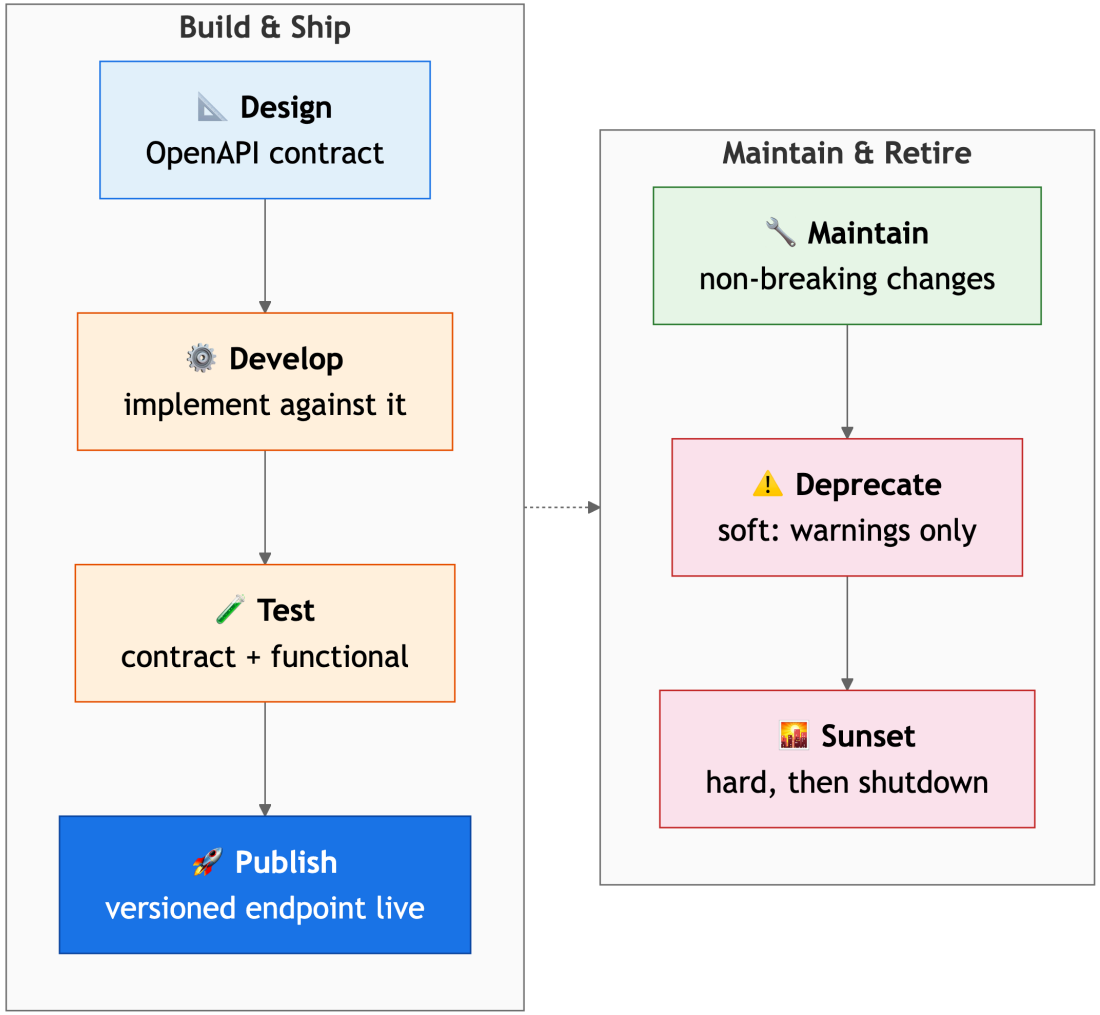
1. Why an API has a lifecycle beyond its launch date
2. Breaking vs non-breaking: the one distinction that matters
3. Versioning strategies for Spring Boot
4. Demo: classifying changes, then routing by version
5. Lab 2.1: designing a non-breaking version strategy

A Version Has a Whole Life

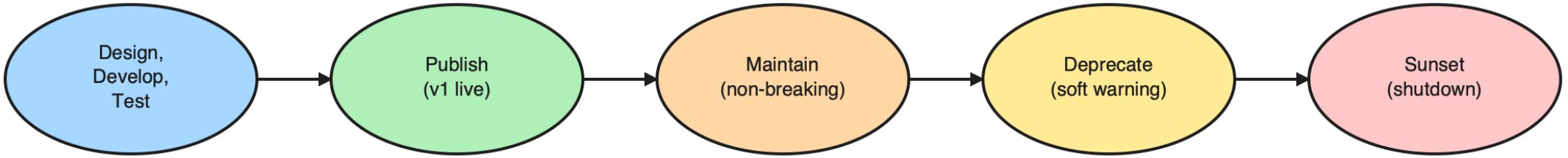
Design is the beginning, not the whole story

KavinSchool

The Lifecycle, Stage by Stage



The Same Arc, at a Glance



A version's whole life, from spec to shutdown

Retirement Is a Process, Not a Switch

1. **Soft deprecation**: old version fully works, a warning is added
2. **Hard deprecation**: old version rate-limited or degraded
3. **Final shutdown**: removed entirely, on a published sunset date

> **CAUTION**
>
> Skipping soft deprecation and cutting a version off without warning is one of the most cited causes of broken partner integrations.

Deprecation: Remove on Friday?

You need to retire `/api/v1/accounts`. The team suggests removing it this Friday.

What should you actually do?

A) Remove it Friday, it's the cleanest cut
B) Add a deprecation warning header first, then rate-limit, then remove on a published date
C) Keep both versions running indefinitely
D) Silently redirect v1 calls to v2

> 💬 **Chat Check**
>
> **Type your answer (A, B, C, or D) in the chat window now!**

Deprecation: The Answer

B, Retirement is a three-phase process, never a switch.

Phase	What happens
Soft deprecation	Old version fully works, a deprecation warning is added
Hard deprecation	Old version is rate-limited or degraded
Final shutdown	Removed entirely, on a published sunset date

Skipping straight to shutdown is one of the most cited causes of broken partner integrations.

Breaking or Non-Breaking?

The only question a version strategy actually answers

KavinSchool

Breaking or Non-Breaking: Discussion

Your API currently returns this JSON: `{"id": 1, "name": "Alice"}`.

You want to change it to: `{"id": 1, "name": "Alice", "role": "admin"}`.

Is this a breaking change or a non-breaking change?

> **DISCUSS**
>
> What is the rule that determines if an API change requires a new version?

Breaking or Non-Breaking: The Checklist

	Non-breaking	Breaking
Fields	Add a new optional response field	Remove or rename an existing field
Endpoints	Add a new endpoint	Change a URL path or HTTP method
Validation	Relax a validation rule	Tighten a rule that rejects previously-valid requests
Params	Add a new optional parameter	Make an optional parameter required

If existing consumer code still works unmodified, it's non-breaking. If it doesn't, it needs a new version.

Breaking or Non-Breaking (Fintech)

REAL-WORLD SCENARIO

Fintech (Navy Federal — Core Banking APIs):

- When Navy Federal adds a new transaction type (like a specific Zelle transfer code), they add it as a new optional field or value without changing existing payloads.
- This non-breaking strategy allows their mobile app to update at its own pace without forcing members on older app versions to immediately upgrade.

Breaking or Non-Breaking (Retail)

REAL-WORLD SCENARIO

Retail (Target — Product Catalog):

- Target's product API added new fulfillment method arrays to existing JSON responses without removing the legacy "in-store" boolean flags.
- Point-of-Sale clients parsing only the boolean flags continued to function perfectly, avoiding a multi-million dollar disruption at checkout.

Breaking or Not?

Your API returns `{"id": 1, "email": "alice@example.com"}`. You change it to `{"id": 1, "email": "alice@example.com", "phone": "555-0100"}`.

What kind of change is this?

A) Breaking, any response body change requires a new version

B) Non-breaking, you added a new optional field, existing consumers still work

C) Breaking, you changed the meaning of the resource

D) It depends on whether the consumer code uses a strict parser

> 💬 **Chat Check**
>
> **Type your answer (A, B, C, or D) in the chat window now!**

Breaking or Not: The Answer

B, Adding a new optional response field is non-breaking.

The rule: if existing consumer code still works unmodified after the change, it's non-breaking. If it doesn't, it needs a new version.

Adding `"phone"` doesn't break any consumer that was already parsing `"id"` and `"email"`, those fields are untouched.

Demo: Running the Checklist

From `day3/demos`, run `java -cp target/classes com.kavinschool.demos.DemoNonBreakingChangeChecklist`.

- Six proposed changes to the shared `/accounts` resource
- Each one classified as breaking or non-breaking, with a reason
- The tally is exactly the judgment call Lab 2.1 asks you to make

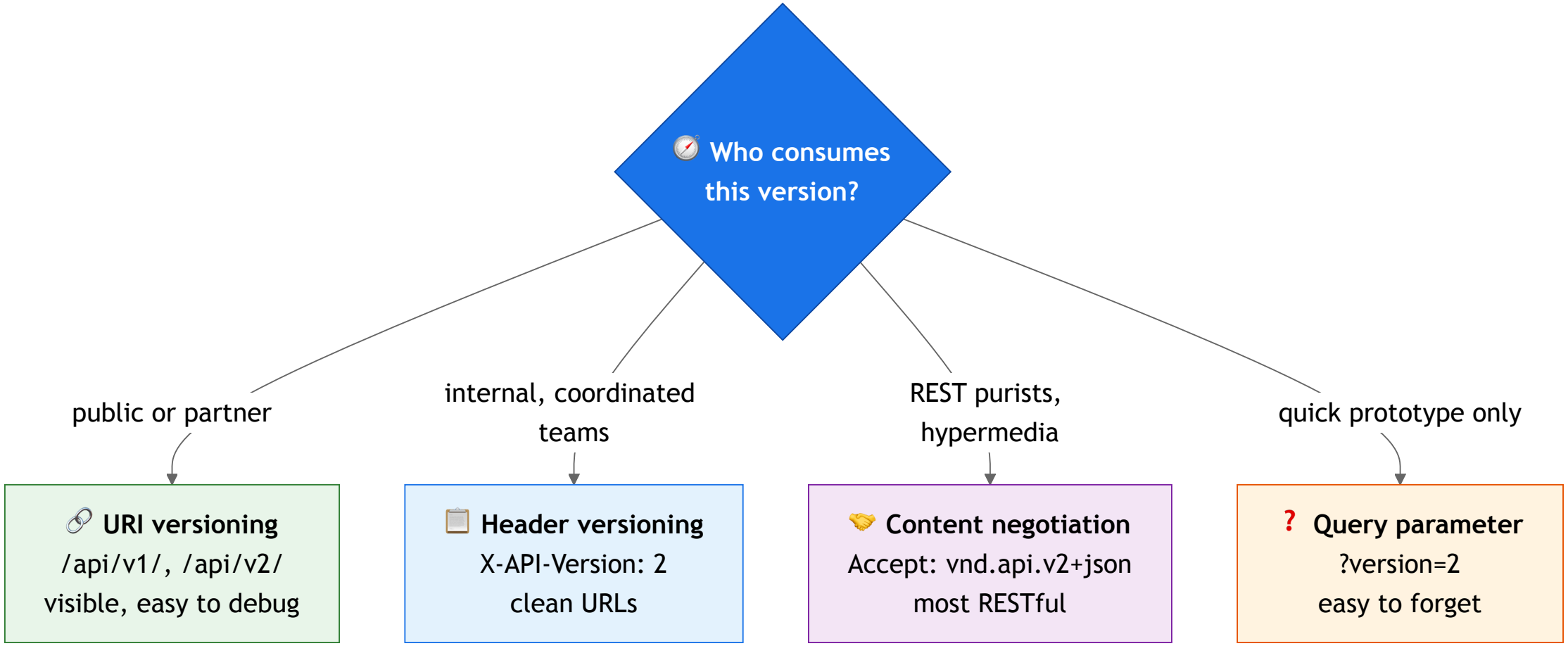
> Offline, no API key, no network.

Choosing a Versioning Strategy

Same decision, four common answers

KavinSchool

Four Strategies, in Spring Boot Terms



The Tradeoffs

Strategy	How	Tradeoff
URI (Uniform Resource Identifier)	`/api/v1/...`, `/api/v2/...`	Visible, easy to debug (most popular for public APIs)
Header	`X-API-Version: 2`	Clean URLs, harder to discover
Content negotiation	`Accept: vnd.api.v2+json`	Most RESTful in theory, least discoverable in practice

NOTE

Real-world enterprises often offload version routing and deprecation blocks to a centralized API Gateway (Kong, Apigee) rather than handling it entirely in application code.

Pick a Versioning Strategy

You're publishing a public banking API for third-party developers you've never met.

Which versioning strategy would you choose?

A) URI (Uniform Resource Identifier) versioning (`/api/v1/...`), visible and easy to debug
B) Header versioning (`X-API-Version: 2`), clean URLs
C) Content negotiation (`Accept: vnd.api.v2+json`), most RESTful
D) No versioning, just keep the API backwards-compatible forever

> 💬 **Chat Check**
>
> **Type your answer (A, B, C, or D) in the chat window now!**

Pick a Versioning Strategy: The Answer

A, URI versioning is the most popular choice for public APIs.

Strategy	Why (not) for public APIs
URI	Visible, easy to debug, anyone can `curl /api/v2/...`
Header	Clean URLs, but harder for third parties to discover
Content negotiation	Most RESTful in theory, least discoverable in practice

When your callers are strangers, visibility beats purity.

Choosing a Versioning Strategy (Fintech)

REAL-WORLD SCENARIO

Fintech (Navy Federal — Partner APIs):

- Navy Federal uses strict URI versioning (`/v1/` , `/v2/`) for APIs exposed to external financial aggregators and partners.
- This visible versioning makes support calls much easier, as the partner can instantly see which contract they are calling just by looking at the URL.

Choosing a Versioning Strategy (Ecommerce)

REAL-WORLD SCENARIO

Ecommerce (Shopify — Admin API):

- Shopify uses a date-based URI versioning strategy (e.g., `/2024-04/orders.json`) for their public Admin API.
- Because their developer ecosystem is massive, visibility and predictability are critical; every partner knows exactly which API release they are bound to.

Demo: URI Versioning in Action

From `day3/demos`, run `java -cp target/classes com.kavinschool.demos.DemoUriVersionRouter`.

- A simulated router dispatches `/api/v1/` and `/api/v2/` to separate handlers
- v2 adds an optional field for new clients
- v1 keeps returning exactly what it always did

> Offline, no API key, no network.

Lab 2.1: Designing a Non-Breaking Version Strategy

Open `day3/lab2_1_designing_a_non_breaking_version_strategy/` in the companion repo.

Goal: Design a version strategy for the shared Spring Boot API that lets it evolve without breaking Day 2's Lab 1.2/1.3 consumers.

1. Pick a versioning strategy and justify it against the public/internal framing
2. Classify a set of proposed changes as breaking or non-breaking
3. Write the deprecation plan for the version being replaced

> Stretch: draft a deprecation/sunset policy; handle a breaking-change edge case.

Key Takeaways

1. An API has a full lifecycle (design, develop, test, publish, maintain, deprecate, sunset), stretching beyond its launch date.
2. The only question that matters for a change is whether existing consumer code still works unmodified.
3. URI versioning is the most common default for Spring Boot; header and content-negotiation versioning fit internal, coordinated consumers.
4. Deprecation is a three-phase process (soft, hard, shutdown), never a switch flipped without warning.

Questions?

Next: Kahoot 1, then a bio break

KavinSchool